



Bridging MDE and AI: a systematic review of domain-specific languages and model-driven practices in AI software systems engineering

Simon Rädler^{1,2} · Luca Berardinelli³ · Karolin Winter⁴ · Abbas Rahimi³ · Stefanie Rinderle-Ma¹

Received: 4 July 2023 / Revised: 1 May 2024 / Accepted: 26 August 2024 / Published online: 28 September 2024
© The Author(s) 2024

Abstract

Technical systems are becoming increasingly complex due to the increasing number of components, functions, and involvement of different disciplines. In this regard, model-driven engineering techniques and practices tame complexity during the development process by using models as primary artifacts. Modeling can be carried out through domain-specific languages whose implementation is supported by model-driven techniques. Today, the amount of data generated during product development is rapidly growing, leading to an increased need to leverage artificial intelligence algorithms. However, using these algorithms in practice can be difficult and time-consuming. Therefore, leveraging domain-specific languages and model-driven techniques for formulating AI algorithms or parts of them can reduce these complexities and be advantageous. This study aims to investigate the existing model-driven approaches relying on domain-specific languages in support of the engineering of AI software systems to sharpen future research further and define the current state of the art. We conducted a Systemic Literature Review (SLR), collecting papers from five major databases resulting in 1335 candidate studies, eventually retaining 18 primary studies. Each primary study will be evaluated and discussed with respect to the adoption of (1) MDE principles and practices and (2) the phases of AI development support aligned with the stages of the CRISP-DM methodology. The study's findings show that language workbenches are of paramount importance in dealing with all aspects of modeling language development (metamodel, concrete syntax, and model transformation) and are leveraged to define domain-specific languages (DSL) explicitly addressing AI concerns. The most prominent AI-related concerns are training and modeling of the AI algorithm, while minor emphasis is given to the time-consuming preparation of the data sets. Early project phases that support interdisciplinary communication of requirements, such as the CRISP-DM *Business Understanding* phase, are rarely reflected. The study found that the use of MDE for AI is still in its early stages, and there is no single tool or method that is widely used. Additionally, current approaches tend to focus on specific stages of development rather than providing support for the entire development process. As a result, the study suggests several research directions to further improve the use of MDE for AI and to guide future research in this area.

Keywords Model-driven engineering · Artificial intelligence · MDE4AI · Domain-specific language · SLR · Literature review · Machine learning

Communicated by Yves Le Traon.

This project has been partially supported and funded by the AIDOaRt project, an ECSEL Joint Undertaking (JU) under grant agreement No. 101007350 and the Austrian Research Promotion Agency (FFG) via the Austrian Competence Center for Digital Production (CDP) under the contract number 881843.

✉ Simon Rädler
simon.raedler@student.tuwien.ac.at

✉ Luca Berardinelli
luca.berardinelli@jku.at

¹ Department of Computer Science, TUM School of Computation, Information and Technology, Technical University of Munich, Boltzmannstr. 3, 85748 Garching, Germany

² Business Informatics Group, Technical University of Vienna, Favoritenstraße 9-11/194-3, 1040 Vienna, Austria

³ Department of Business Informatics-Software Engineering, Johannes Kepler University, Altenberger Straße 69, 4040 Linz, Austria

1 Introduction

Engineering systems are becoming more complex due to the increasing number of components, functions, and the involvement of several disciplines [7]. To address this complexity, the integration of model-driven engineering (MDE) is promising [44, 46, 58].

MDE aims to support the engineering of systems by providing and maintaining information for the development using models rather than documents [44]. Models are enriched with information, shared among stakeholders, and manipulated by Computer-Aided Software Engineering (CASE) tools, aiming at the highest possible degree of automation, e.g., via model transformations. Consequently, MDE techniques allow informed decisions by producing and consuming models as machine-processable artifacts.

Although the models are enriched with information, MDE techniques lack the means to gather sufficient knowledge from more extensive data, e.g., Big Data or data collected from complex (software) systems. In this respect, means of artificial intelligence (AI) and its subdisciplines, namely machine learning (ML) and deep learning (DL), are beneficial to exploit the information hidden in data [69]. Integrating data-driven methods to support engineering tasks has recently been defined as data-driven engineering [84]. It has proven to be beneficial in several engineering areas such as manufacturing [31], aerospace industry [17] or other industrial applications [10, 34, 79].

AI integration is mainly case-specific, and thus there are several methods supporting the implementation of AI in literature [4], e.g., Cross Industry Standard Process for Data Mining (CRISP-DM) [87], which support the development of AI tools by providing support for the specific development steps typically applied to AI projects. The incorporation of methods such as CRISP-DM and MDE capabilities is rarely considered in the literature, even though the need for experts to implement data-driven solutions is increasing due to the requirement to integrate AI into existing methods [25] and the implementation effort is decreasing by applying MDE principles and practices to the development of AI capabilities.

In recent times, a series of workshops has been initiated, focusing on the intersection of model-driven engineering (MDE) techniques and artificial intelligence (AI) [19, 20, 22]. Specifically, these workshops explore the use of MDE practices to define AI methods (referred to as MDE4AI) and to provide AI support for MDE (AI4MDE). The goal is to enhance software development by automating engineering activities through techniques like code generation. The current state of practice and state-of-the-art MDE approaches

Table 1 The overall research goal

Purpose	Collection and comparison of studies on
Issue	model-driven approaches leveraging suitably designed domain-specific languages that explicitly address the engineering of
Object	software systems leveraging AI/ML
Viewpoint	from the point of view of researchers

that facilitate the implementation of AI capabilities remain insufficiently explored.

In this context, the overall research goal (RG) of our systematic study can be defined as presented in Table 1. The definition of the RG aligns with the Goal/Question/Metric perspective, as proposed by Basili et al. [5].

We define the following overarching research question (RQ) based on this research goal:

Main RQ What is the current state of the art for model-driven engineering with extensions to formalize artificial intelligence methods and applications?

To address the main research question, various refined and more fine-grained RQs are introduced in Sect. 3.

For this reason, we conducted a systematic literature review according to the guidelines set out by [51] to address the identified research questions [65] and spotlight model-driven approaches that leverage suitably designed DSLs to automate the engineering of systems with AI capabilities, specifically emphasizing MDE4AI, and ii) the support to data mining steps. Accordingly, AI techniques for enhancing MDE approaches fall outside the scope of our work.

The contributions of this SLR comprise:

- Collection and analysis of state-of-the-art model-driven approaches adopting specialized DSLs for AI applications in (software) system engineering.
- Quantitative assessment criteria for model-driven approaches for AI.
- Quantitative assessment of existing approaches with derived research opportunities.

The remainder of the paper is organized as follows. Section 2 provides background on terms such as MDE and AI. Section 3 introduces the research methodology, i.e., the paper search and selection process. Section 4 presents the approaches aligned with the data extraction strategy of the SLR protocol in Sect. 3. Section 5 answers the research questions, discusses the key findings, and depicts implications and future research. Section 6 assesses the quality and limitations of the current SLR using threats to validity analysis. Finally, Sect. 7 summarizes the paper.

⁴ Department of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Groene Loper 3, 5612 AE Eindhoven, The Netherlands

2 Background

This section presents relevant background on MDE and AI. The focus is on fundamentals that support understanding of the SLR results and is not intended to reflect the current state of the art in each research area.

2.1 Model-driven engineering

The core of MDE includes the pillar concepts of *model*, *meta-model*, and *model transformation* [13, 74].

Models are machine-readable artifacts representing particular concerns of a system under study, such as design-time information like software architecture or hardware platform, or operational information like monitored data. *Metamodels* define the modeling concepts and their relationships, providing an intentional description of all possible models that must conform to the associated metamodel. From a language engineering perspective, a metamodel represents the *abstract syntax* of a modeling language. *Metamodels* define modeling languages conceptually and are independent of any concrete representation. The *concrete syntax* of a language assigns graphical or textual elements to metamodel elements that can be understood by users and edited through model editors [13, 74]. As models in MDE are considered machine-readable artifacts, so-called *model transformations* apply to modifying existing or generating new engineering artifacts. These artifacts are then used for particular purposes. During the development process, models support realizing the steps of the envisioned engineering process toward the (partial or full) generation of the software system. At runtime, models provide intelligent support during execution. According to [8], ‘a runtime model is defined as a casually connected self-representation of the associated system that emphasizes the structure, behavior, and goal of the system and which can be manipulated at runtime for specific purposes.’

Depending on the specific engineering concerns (software, hardware, or system as a whole) as well as the role played by model artifacts due to degrees of automation of model management activities (e.g., model-based being to refer to a lighter version of model-driven), different modeling acronyms are typically used (e.g., MBE, MBSE, MDE, MDSE, MDD, MDA).¹ The various modeling acronyms show that the MDE community is widespread, and the same applies to the goals and applications of MDE approaches.

Recently, so-called low-code and no-code development platforms gained the attention of researchers and the market [29]. LCDPs leverage MDE principles by utilizing automation, analysis, and abstraction through modeling and metamodeling [76]. One of the goals of these platforms is to offer

degrees of automation in the generation of software applications while partially or completely hiding code to their users, typically referred to as citizen developers, i.e., users with limited programming experience in the software development process.

2.2 Artificial intelligence

Artificial intelligence is a flourishing science with numerous practical applications, ranging from image/voice recognition to recommendation systems and self-driving cars. The primary goal of AI is to tackle problems that are tough for humans but relatively simple for computers [39]. In a non-scientific context, the different terms around AI are quite fuzzy, and depending on the application area, different terms like machine learning (ML), deep learning, data science, data mining, and so on are used exchangeable [32]. In science, some of the terms are today used interchangeable, while in computer science, the terms are precisely defined. For example, *data science* is defined as the umbrella term referring to the broad field of extracting information and knowledge by analyzing data to derive patterns, trends, etc., and report them as human-understandable insights [78] that are beneficial for various areas [82], such as manufacturing [31]. Another sample is data mining [69], which is the extraction of knowledge from datasets or systems and processes.

Although each term refers to a specific subcategory of data science, the implementation phases are essentially similar. Therefore, in the following, all synonyms and sub-terms will be related to the term AI for better understanding. Consequently, methodologies have been developed to structure and support the implementation of AI projects [4, 33, 35, 87, 89]. The implementation phases of the methodologies in literature are quite similar, although the naming is different [4]. In this work, the focus is on the phases of the CRISP-DM methodology [87]. The main reason for orienting to CRISP-DM is that it is described in the literature as a de-facto standard in the industry and is widely used due to its generality [77, 81]. Additionally, the phases of CRISP-DM can be applied to other sub-topics of data science projects that are not covered under the term data mining. CRISP-DM comprises six phases [87]:

1. The *business understanding* phase involves gathering knowledge on the application domain and the project objectives.
2. The *data understanding* starts with initial data collection and analysis to get familiar with the data.
3. In the *data preparation* phase, the input datasets are built from the given data by applying techniques such as normalization to transform the data.

¹ See <https://modeling-languages.com/clarifying-concepts-mbe-vs-mde-vs-mdd-vs-mda/> for a discussion.

4. The *modeling phase* applies the AI algorithm to the prepared data, and additional fine-tuning is applied, e.g., hyper-parameter optimization.
5. The *evaluation phase* is used to obtain whether the elaborated model performs as it should.
6. The *deployment phase* deals with infrastructure and the presentation of customer-usable knowledge.

2.3 Related work

Relevant secondary studies started investigating the combination of model-driven software engineering techniques and AI/ML.

In their studies, both Giray [38] and Martínez-Fernández et al. [59] investigate Software Engineering (SE) practices for AI-based systems, or SE4AI. In particular, [38] highlights the shift from traditional software development to ML systems that learn from data, necessitating reevaluating software development practices. The study systematically reviews 141 studies to analyze the current research on software engineering (SE) for ML systems, revealing that none of the SE aspects have mature tools and techniques, with testing being the most researched area. The paper identifies the need for experiments and case studies in industrial settings to understand challenges and propose solutions, aiming to assist practitioners, researchers, and educators in ML systems engineering. Giray highlights how the nondeterministic nature of ML systems complicates SE, the lack of mature tools, and the importance of practical research to advance the field.

In [59], the research highlights the field's significant expansion since 2015, with a notable focus on dependability and safety. The authors have meticulously mapped prevalent SE methodologies across the areas outlined in the Software Engineering Body of Knowledge (SWEBOK[12]). Despite the work's substantial contribution to bridging the SE and AI disciplines, it points out a notable gap: MDE remains underexplored within SE for AI-based systems. Yet, MDE is acknowledged for its potential to handle complexity and ensure uniformity across various models and artifacts, culminating in more resilient and maintainable software solutions. With respect to [38, 59], our research delves deeper into the role of MDE in the context of AI-based systems, referred to as MDE4AI. Specifically, we focus on DSLs utilized within the engineering process and their support in facilitating data mining activities, and we analyze their effectiveness from the perspective of the CRISP-DM framework.

To refine the focus from the broader domain of SE4AI to the more specialized field of MDE4AI, we found two surveys that have contributed to this area of research.

In [68], Portugal et al. surveyed DSLs and frameworks for designing machine learning algorithms in the context of Big Data. The DSLs were categorized based on the classification proposed by [23, 36, 85]. However, it is important to note

that their survey lacked a systematic approach; no explicit surveying protocol was followed. In contrast, our approach introduces a classification based on DSL engineering principles and practices commonly used in MDE [13, 21]. We emphasize the relationship between DSLs and the implementation phases of AI algorithms, which was not explored by Portugal et al. Additionally, we acknowledge and utilize the referenced literature from Portugal et al. in our survey for snowballing purposes.

The second survey by Naveed et al. [64] is the most interesting and delve into the intersection of MDE and machine learning components, which they term MDE4ML. Their study runs in parallel with our systematic review,² and like ours, it aims to analyze existing research on MDE4ML (a subset of MDEAI). The authors' review process results in the selection of 46 primary studies, highlighting the increasing adoption of ML as a component in various software applications. This trend is driven by the desire to harness large volumes of data for predictive capabilities and informed decision-making. Naveed et al.'s study also explores a range of MDE solutions applied to ML systems, including modeling languages, model transformations, and tool support. Notably, the corpus of selected studies partially overlaps with ours [2, 6, 11, 37, 41, 52, 72], emphasizing the importance of investigating the integration of MDE and AI/ML in software and system engineering approaches. While both studies provide an overview of MDE techniques and practices within the MDE4AI domain, our SLR places particular emphasis on explicitly modeling AI/ML concerns in DSLs and the intent behind related model transformations. Additionally, our analysis of AI/ML support for data mining aligns with the CRISP-DM standard. In summary, our work provides a complementary viewpoint and results, contributing to the broader understanding of MDE and AI/ML integration in software engineering.

In [18] Bucaioni et al. discuss the rise of low-code development (LCD), supporting LCDPs, and its growing interest in the software engineering community, particularly in MDE, presenting a multi-vocal systematic review. Their review highlights the increasing publication trend in low-code research, the prominence of MDE as a core technology in LCD, and the wide range of business domains where LCD is applied. However, the adoption of LCD for developing AI-augmented software systems is still in the early stages. Only one surveyed paper [53] provides insights into the architectural aspects that leverage MDE for monitoring machine learning model performance. However, it does not delve into specific DSLs that could be employed within such a platform. The study by Bencomo et al. [8] discusses the principles and requirements of models at runtime and the state of the art. In this context, machine learning techniques have been reported

² The preprint of this work has been cited by [64].

to support reasoning on uncertainty, building context-aware systems, systems-of-systems, and self-aware systems, and extracting information at runtime to dynamically build the models. However, adopting AI/ML is still limited and recognized as a challenge in applied model-driven techniques.

3 Research methodology

This section introduces the SLR method applied in this work. The SLR study protocol is based on the guidelines by [50, 51, 65], introducing the main steps of SLRs to be performed in the Software Engineering domain.

Figure 1 depicts an activity-like diagram of the performed search and selection process protocol workflow. The workflow consists of the following steps:

1. *Identifying the research goals and the research questions (RG/RQ)*: The objective of this work and the research questions are defined to guide the SLR (Sect. 1 shows the result of the RG/RQ elaboration)
2. *Search process*: The literature search is conducted on selected databases collecting scientific publications via the execution of queries based on a search string suitably designed according to the given RGs and RQs (Sect. 3.2).
3. *Study selection*: The authors define the inclusion and exclusion criteria (IC/EC) and apply them to the papers collected in the databases by reading their titles and abstracts. Subsequently, the selected papers are evaluated based on their content (Sect. 3.3).
4. *Data extraction*: Detailed information is collected from selected studies that have passed the inclusion and exclusion criteria (IC/EC). This extraction occurs during a thorough full-text reading. The collected data are organized into evaluation tables. Additionally, backward and forward snowballing techniques are applied to selected studies.
5. *Results analysis and discussion*: Collected results are analyzed, and a discussion occurs among the authors to answer the stated RQs.

The execution of the protocol is documented in a spreadsheet,³ and bibliographic entries are collected in Zotero Library. An export can be found online³.

3.1 Research questions

The overall research goal has been previously introduced in Table 1, aligning with the main research question. To address

the main research question, we have defined several refined and more fine-grained research questions as follows :

RQ1 What are the prevalent model-driven engineering (MDE) concepts and practices [13, 21] being applied in current studies, such as metamodeling and model transformations?

This research question aims to evaluate the application and evidence of MDE concepts and practices.

RQ2 Which phases of AI development aligned with the CRISP-DM methodology are covered by the approaches?

This RQ assesses the extent to which the development phases of CRISP-DM are covered. As a result, implications can be made about the extent of support.

RQ3 Which application domains actively incorporate model-driven engineering (MDE) methodologies in AI applications?

This RQ aims to identify the specific application domains that actively incorporate MDE methodologies in AI applications. The goal is to understand if any predominant and leading application domain is leading and shaping the evolution of MDE4AI.

RQ4 What are the used methods and the supporting MDE tools the proposed approaches rely on?

This RQ allows assessing the underlying methods and the related tool support, including further development leveraging these underlying technologies to gain maturity.

RQ5 To what extent is communication between different stakeholders supported by MDE?

Communication and business knowledge elaboration are two of the core pitfalls in the development of AI solutions [67]. Therefore, this question aims to assess the contribution to support fostering AI in the industry.

RQ6 Which challenges and research directions are still open?

This RQ will lead to future research directions and challenges for MDE4AI applications due to a collection of limitations in the proposed approaches based on respective authors or our obtainment.

3.2 Search process

This section describes the search activity in Fig. 1. According to [51], defined search queries are executed on dedicated search engines. In this research, the queries are performed on the following bibliographic sources:

- ACM Digital Library: <http://dl.acm.org/>
- dblp Computer Science Bibliography: <https://dblp.org/>
- IEEE Xplore Digital Library: <http://ieeexplore.ieee.org>
- Google Scholar: <https://scholar.google.com>

³ <https://github.com/sraedler/Model-Driven-Engineering4Artificial-Intelligence>

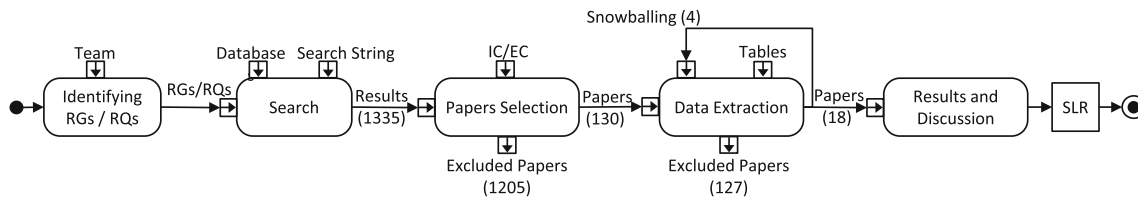


Fig. 1 SLR methodology overview

- Springer: <https://link.springer.com>

To select suitable terms for the search, keywords from known studies, the MDE4AI⁴ and Low-Code⁵ workshops, and the International Journal on Software and Systems Modeling (SoSyM)⁶ were selected.

The selected keywords for the search terms are the following:
 $S_1(MDE) = \{MDE; Model - Driven Engineering; Model - Driven Development; DSL; Domain Specific Language; Metamodeling; Domain Modeling, Low - Code; No - Code; Models@Runtime; Runtime Models\}$

$S_2(AI) = \{AI; Artificial Intelligence; ML; Machine Learning; DL; Deep Learning; Neural Network; Data Science; Intelligence; Data Analytics\}$

Each keyword k_i from the sets S_1 and S_2 has been combined in conjunctive logic proposition $p \in P$.

$$P = \{p | p = s_i \in S_1 \wedge s_j \in S_2\}$$

$$i = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12$$

$$j = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$$

The resulting set P of 120 propositions (p_i) includes the final *search strings*. According to [51], the propositions (p_i) should be combined as OR statements. However, for some search engines, a single search term is too complicated, as some search engines limit the length of the search term or do not generate results correctly due to nested search terms. Therefore, each search string is executed as a single query. The automated search was executed in November 2022 and in February 2024 again. In total, 1335 papers have been collected. The search terms and results are archived and are online available³. If a result file is unavailable, the search query on the specific search engine did not retrieve any results.

⁴ <https://mde-intelligence.github.io/>

⁵ <https://lowcode-workshop.github.io/>

⁶ <https://www.sosym.org/>

3.3 Paper selection

The inclusion and exclusion criteria (IC/EC) outlined in Table 2 are employed for the paper selection. The IC/EC have been evaluated for each paper collected by queries executed on the selected databases by reading its title and abstract.

We establish two inclusion criteria for selecting papers. The first one is intended to select papers based on an MDE standpoint. We focus on work related to MDE4AI. Our inclusion criteria encompass papers that explicitly present DSLs defining AI/ML extensions as first-class language concepts. These DSLs should allow the specification elements related to AI/ML algorithms like inputs, outputs, parameters, or hyper-parameters.

We exclude DSLs that merely provide a link to AI/ML artifacts, even if they fall within the field of model-driven engineering for artificial intelligence (MDE4AI) (e.g., [3]). The second one is intended to select papers based on an AI/ML standpoint. We explicitly exclude any paper adopting AI/ML learning techniques to further enhance MDE approaches (AI4MDE) like modeling recommender based on AI/ML algorithms (e.g., [30]). We also omit studies focused solely on DSLs that specify algorithms, such as data science workflows (e.g., [28]), unless these DSLs are integrated within a broader AI-augmented software system.

Ultimately, adhering to standard exclusion criteria SLRs, we omitted all secondary studies, non-English publications, vision statements, nonacademic articles, proposals, and theses.

Following the IC/EC criteria application, a full-paper read is applied to select the final papers. Additionally, as suggested by [51], snowballing is accomplished to retrieve further results. The relevant papers from the list of forward/backward snowballing papers were selected using the same procedure as the query results. Table 3 lists the final list of selected papers aligned with the publication venue. Particularly, 18 papers are added by query selection [2, 9, 11, 37, 41, 42, 45, 52, 60, 62, 63, 66, 71, 72, 80] and four are added due to snowballing [26, 43, 55, 61].

Table 2 Inclusion criteria (IC) and Exclusion criteria (EC)

Type	ID	Type
IC	1	The paper presents a DSL with AI extensions
	2	The paper leverages MDE body of knowledge, techniques, and practices [13, 21]
	3	Full text of the paper is available
EC	1	We exclude papers with focus on AI for MDE
	2	We exclude MDE approaches whose DSLs do not include any AI concept as first-class entities
	3	We exclude non-primary studies
	4	We exclude studies available only in the form of abstract
	5	We exclude study not in English language
	6	We exclude vision only, nonscientific papers, and proposals
	7	We exclude any thesis

Table 3 List of selected publications

Paper type	Venue	Year	Author	Title
Conference	International Conference on Intelligent Data Science Technologies and Applications (IDSTA)	2020	[2]	Model-Driven Approach for Neural Networks
Congress	IEEE Congress on Evolutionary Computation (CEC)	2023	[9]	EVOAL: A domain-specific language based approach to optimisation
Conference	IEEE International Conference on Big Data (Big Data)	2019	[11]	STRATUM: A BigData-as-a-Service for Lifecycle Management of IoT Analytics Applications
Journal	Journal of Computer Languages	2020	[26]	Lavoisier: A DSL for increasing the level of abstraction of data selection and formatting in data mining
Conference	International Conference on Model-Driven Engineering Languages and Systems (MODELS)	2022	[37]	A domain-specific language for describing machine learning dataset
Journal	Software and Systems Modeling (SoSym)	2019	[41]	The next Evolution of MDE: A Seamless Integration of Machine Learning into Domain Modeling
Conference	International Conference on Model-Driven Engineering Languages and Systems (MODELS)	2019	[42]	Meta-Modelling Meta-Learning
Workshop	Workshop on Design Automation for CPS and IoT (DESTION)	2019	[43]	Model-based design for CPS with learning-enabled components
Journal	IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems	2023	[45]	AIoTML: A Unified Modeling Language for AIoT-Based Cyber-Physical Systems
Conference	International Conference on Model-Driven Engineering and Software Development (MODELSWARD)	2019	[52]	Realization of a Machine Learning Domain-Specific Modeling Language: A Baseball Analytics Case Study
Conference	International Conference on Automated Software Engineering Workshop (ASEW)	2019	[55]	On the Engineering of AI-Powered Systems
Journal	IEEE Access	2021	[60]	AdaptiveSystems: An Integrated Framework for Adaptive Systems Design and Development Using MPS JetBrains Domain-Specific Modeling Environment
Conference	International Conference on Advanced Information Systems Engineering (CAISE)	2022	[61]	A Model-Driven Approach for Systematic Reproducibility and Replicability of Data Science Projects
Journal	Software and Systems Modeling (SoSym)	2021	[62]	A Model-Driven Engineering Approach to Machine Learning and Software Modeling

Table 3 continued

Paper type	Venue	Year	Author	Title
Journal	International Conference on Product-Focused Software Process Improvement (PROFES)	2022	[63]	Towards a DSL for AI Engineering Process Modeling
Journal	IEEE Access	2023	[66]	RADENN: A Domain-Specific Language for the Rapid Development of Neural Networks
Conference	Conference Modellierung	2023	[71]	Code Generation for Machine Learning using Model-Driven Engineering and SysML
Conference	International Conference on Model-Driven Engineering and Software Development (MODELSWARD)	2021	[72]	An MDE Method for Improving Deep Learning Dataset Requirements Engineering using Alloy and UML

3.4 Data extraction

Each selected paper presented in Table 3 underwent a data extraction process following the data extraction template in Table 4. Additionally, the publication type is assessed as Exploratory (without evaluation, e.g., a pure concept or vision) or Technical (with evaluation).

The extracted data mainly address two concerns of interest, i.e., MDE and AI. Modeling concerns refer to the evidence of sound knowledge and application of *model foundations* [21] (e.g., abstract syntax/grammar/metamodel, textual/graphical concrete syntax, constraints, model transformations) and supporting tools (e.g., modeling language frameworks). AI concerns [1] indicate to which extent the publications support ML modeling aligned with the dimensions of the CRISP-DM methodology [87]. It should be noted that the assessment dimensions do not correspond exactly to the phases of CRISP-DM to allow for a more detailed categorization of concerns; e.g., in CRISP-DM, *Data Ingestion* is part of the *Data Understanding* phase but separated in the given assessment. An aspect of a concern of interest is assessed as *available* (■) if the aspect is presented in the approach or as *underlying principle* if it is typically offered by the underlying environment (e.g., constraint modeling might not be presented but is typically offered by the underlying MDE tooling). Finally, it is worth noting that there is no evaluation of the deployment phase of CRISP-DM as it is beyond the scope of this paper.

4 Literature assessment

The result obtained from the data extraction process described in the previous section is presented in Tables 5, 7 and 8.

In [2], Al-Azzoni proposes a model-driven approach to describe ML problems addressed by artificial neural networks. The approach enables the description of datasets as well as the consuming multilayer perception (MLP) neural networks (NN). With templates and code generators, exe-

cutable Java programs can be generated. The approach is validated using the Pima Indians Diabetes dataset. Future work of the approach consists of supporting a wider variety of NNs and support for code generation.

In [9], Berger et al. present three DSLs allowing to structure requirements for optimization projects relying on evolutionary algorithms. The approach separates and references formalized knowledge of various experts to enable functional and nonfunctional requirements definition in a textual set of DSLs. However, the integration of hyper-parameter tuning is not given but planned in future.

In [11], Bhattacharjee et al. introduce STRATUM, a model-driven tool that enables dealing with the lifecycle of intelligent component development. The platform addresses design-related concerns such as modeling the ML algorithm pipeline, accessing data streams, allocating and properly sizing cloud-based execution platforms, and monitoring the overall system's quality of service. The primary goal of this work is to support deploying and maintaining various cloud-based execution platforms. The MDE part of this work is minor and less detailed.

In [26], De La Vega et al. introduce a DSL that describes datasets to select sufficient data on a high level. The approach uses a SQL-like textual language to select, combine and filter various data on an attribute level. The approach aims to increase a dataset's abstraction level to reduce complexity and make using data mining technologies easier. The outline of future work indicates that the authors want to perform some empirical experiments to assess the approaches usability, learning curve, and effectiveness. Additionally, the extension of the approaches to be applicable in a wider area of application is intended.

In [37], the DescribeML DSL is proposed to define ML datasets. From a DescribeML model, a template with basic information is automatically generated, based on a given dataset. The provided DSL allows the definition of metadata, data attributes with statistical features and provenance, and social concerns. This approach aims to improve the under-

Table 4 Data extraction template

RQ	Concern	Assessment description
RQ1	MDE	Language Workbench (LW)
		Metamodel(s)/Grammar(s) (MM/G)
		Concrete Syntax(es) (CS)
		Validation (V)
		Model Transformation(s) (MTrafo)
		Development and Runtime Model (D/R)
RQ2,RQ5	AI	Business Understanding (BU)
		Data Understanding (DU)
		Data Ingestion (DI)
		Feature Preparation (FP)
		Model Training (MTrain)
		Metrics/Evaluation (ME)
RQ3	Others	Problem Domain
RQ4		Frameworks

standing of datasets and thus support the replicability of AI projects. Currently, this work is limited to the dataset description. Future work aims to describe AI models and other elements of an AI pipeline and integrate with common web browsers.

In [41], Hartmann et al. present an approach based on so-called microlearning units at a language definition level. This work proposes to weave the learning units into domain modeling, due to the high entanglement of learning units and domain knowledge. For this purpose, the approach allows the definition of DSLs with learned attributes (i.e., what should be learned), how (i.e., algorithm and parameters), and from what (i.e., other attributes and relations).

Hartmann et al. leverage the previous study for meta-learning in [42]. This study proposes two generic metamodels for modeling i) ML algorithms and ii) meta-ML algorithms (i.e., algorithms to learn ML ones). Future work of the authors is to support a wider range of algorithms and parameters.

In [43], a comprehensive modeling environment for learning-enabled components in CPS development is introduced. The approach supports training, data collection, evaluation, and

verification. It integrates goal structuring notation (GSN) to support assurance and safety cases. The publication is, among others, part of a research project⁷ facilitating MDE. Currently, the approach is only simulation-based. Therefore, future work consists of integrating hardware-in-the-loop to enable verification.

In [45], Ming et al. propose a novel modeling language, AIoTML, to facilitate the development of cyber-physical systems (CPS) that leverage the power of artificial intelligence of things (IoT). The proposed AIoTML DSL extends ThingML, preserving compatibility and introducing new language constructs to define new datatypes and AI strategies based on deep learning and reinforcement learning models and mechanisms for physical modeling and simulation, enabling the construction of digital twins and optimization of control strategies for CPSs.

In [52], a DSL is introduced with the goal of proving the plausibility of using MDE approaches to create ML software. The DSL, conceptually sketched by another research group

⁷ <https://modelbasedassurance.org/>

Table 5 Result of the data extraction for the MDE and AI concerns

RQ	Concern	Paper	[2]	[9]	[11]	[26]	[37]	[41]	[42]	[43]	[45]	[52]	[55]	[60]	[61]	[62]	[63]	[66]	[71]	[72]	Sum
RQ1	General	Type	T	T	T	T	T	T	E	T	T	T	T	T	T	T	E	T	T	T	15 T 3 E
	MDE	LW	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	17
		MM/G	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	18
		CS	xmi	txt	g	txt	txt	txt	txt	g	txt	g	txt/g	txt	xmi	txt	g	txt	g	g	10 txt 7 g
		V	■	■											■						2 xmi 4
RQ2, RQ5	AI	MTrafo	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	15
		D/R	d	d	d	d	d/r	d	d	d	d	d	d	d	d	d	d	d	d	d	18 d 1 r
		BU								■	■						■		■	■	5
		DU		■	■	■	■	■	■			■					■	■	■	■	11
		DI	■		■	■				■			■	■	■	■	■	■	■	■	11
		FP	■								■				■	■	■	■	■	■	8
		MTrain	■					■		■	■	■	■	■	■	■	■	■	■	■	14
		M/E								■						■		■	■	■	6

Table 6 Model transformation intent category and concrete intent

Paper	Intent category	Concrete intent	Tool
[2]	Refinement	Synthesis (model to code)	Epsilon EGL
[11]	Refinement	Synthesis (model to code)	JS Implementation
[26]	Language Translation	Translation	Xtend
	Abstraction	Restrictive Query	
[37]	Language Translation	Translation	Typescript (Visual Studio Code)
[41]	Refinement	Synthesis (model to code)	KMF/GreyCat
	Semantic Definition	Translational Semantics	
[42]	Refinement	Refinement	n.a.
	Refinement	Synthesis (model to code)	
[43]	Refinement	Synthesis (model to code)	n.a.
	Semantic Definition	Translational Semantics	
	Analysis	Safety Analysis	
[45]	Refinement	Synthesis (model to code)	compilers written in Java
[52]	Refinement	Synthesis (model to code)	Epsilon EGX/EGL/EOL
[55]	Refinement	Synthesis (model to code)	EmbeddeMontiArc/EMADL2CPP
[60]	Refinement	Synthesis (model to code)	JetBrains MPS
	Model Composition	Model Merging	
[62]	Refinement	Synthesis (model to code)	Xtend
[66]	Language Translation	Translation	Python
[71]	Refinement	Synthesis (model to code)	Python
[72]	Semantic Definition	Translational Semantics	Xtend

in [14], is realized and applied to a case study in the sports domain. The approach integrates model transformation to generate executable code. Future work consists of extending the approach to other use cases and conduct empirical studies. In [55], an approach describing deep learning using MDE is presented. The approach combines two DSLs, namely, MontiAnna and EmbeddedMontiArc. The former is a textual modeling framework for designing and training artificial neural networks (ANNs). It also embeds another DSL, MontiAnnaTrain, for describing the training procedure. The latter, EmbeddedMontiArc, is an architectural description language. It supports the definition of components and connectors, with a particular focus on embedded, automotive, and cyber-physical systems. The frameworks are intended to define deep artificial neural networks, e.g., convolutional neural networks, for processing traffic images to learn how to drive a car in a simulator. At this point, it should be noted that only the journal publication by Kusmenko et al. was taken into account, although there are several comparable or extension approaches in the literature list of the author and the associated research group.⁸ The reason for this is that the MDE dimension is overall the same and only small changes in the AI assessment, such as support for different algorithms, are shown.

In [60], Meacham et al. propose a set of DSLs and toolset implemented on top of the MPS⁹ language workbench for the design and development of adaptive systems offering MAPE-K and AI in context capabilities. The approach describes an extension and composition of DSLs that are extended with application-specific concepts. Future work consists of extending the range of domains the approach is applicable. In [61], Melchor et al. propose an MDE approach to formalizing ML projects and the associated infrastructure in which the resulting tool will be deployed. The approach aims to increase the reproducibility and replicability of data science projects. Hence, a key feature of the approach is to describe processes and datasets in detail. With respect to this, future work aims to add verification on a model level to support this core competence.

In [62], Moin et al. present an MDE approach based on ThingML¹⁰ to support the development of IoT devices with the extension of data analytics and ML. The ThingML framework supports defining software parts and components using UML. The communication between the components (things) is defined using ports, messages, and state machines. The approach supports the transformation of the model into executable code. Future work aims to extend the approach to more methods, to other technologies such as semi-supervised

⁸ <https://scholar.google.de/citations?hl=de&user=bUjXPauAAAAJ>

⁹ Meta Programming System.

¹⁰ <https://github.com/TelluIoT/ThingML>

Table 7 Used methods and tools (RQ4)

Tools or method	Cat.	Type	[2]	[9]	[11]	[26]	[37]	[41]	[42]	[43]	[45]	[52]	[55]	[60]	[61]	[62]	[63]	[66]	[71]	[72]	Total
Alloy	<i>Other</i>	Lang																		■	1
BPMN	MDE	Lang																		■	1
DeepForge [43]	AI/ML	Tool							■												1
EMF	MDE	LW	■			■									■				■		5
Epsilon	MDE	Tool	■									■									2
GSN [49]	MDE	Lang							■												1
GreyCat	MDE	LW					■		■												2
JetBrains MPS	MDE	LW												■							1
Jupyter Notebook	AI/ML	Tool							■										■		2
Langium	MDE	LW					■													■	2
MontiCore	MDE	LW											■								1
Papyrus	MDE	Tool										■							■		3
Python	AI/ML	Lang							■										■		3
ROSMOD [54]	MDE	Tool								■											1
SEAM ¹⁹	<i>Other</i>	Std								■											1
SQL	<i>Other</i>	Lang																	■		1
Sirius	MDE	Tool							■												2
ThingML [40]	MDE	Lang										■									2
WebGME	MDE	LW			■					■											2
Xtext	MDE	LW				■														■	3
<i>Total</i>			2	1	1	3	1	1	2	6	1	2	1	1	1	3	2	1	4	6	

<https://standards.nasa.gov/standard/nasa/nasa-std-87291>

Table 8 Availability and type of artifacts aligned with the type of application

Selected study	In the study	Online (Git, Server)	No artifacts	Type of application
[2]	■			Datasets
[9]		■		Datasets
[11]		■		IoT
[26]		■		Datasets
[37]		■		Datasets
[41]		■		IoT
[42]		■		Unknown
[43]		■		IoT (CPS)
[45]		■		IoT
[52]		■		Datasets
[55]		■		IoT (Image)
[60]			■	Adaptive Systems
[61]		■		Datasets
[62]		■		IoT
[66]		■		Datasets
[63]	■			Unknown
[71]		■		IoT (Datasets)
[72]		■		Datasets

learning, and to enable the generation of various other target languages.

In [63], Morales et al. provide a DSL to model AI-related processes using Eclipse-based technologies. The approach aims to describe AI processes within an organization and thus contribute to the structured designing, enacting, and automating of AI engineering processes. Future work consists of extending the approach to be connectable, e.g., with the domain needs.

In [66], Pineda et al. present an imperative DSL calls RADENN, allowing to model deep learning using a textual syntax. The approach defines a grammar constituted by 27 rules with various built-in functions such as *general* functions like ‘print’ or ‘save,’ *dataset* functions enabling to load and store datasets and *network* functions related to neural network functions like train, predict, or evaluate. The approach aims to enable deep learning modeling with a focus on simplified syntax and increased efficiency while maintaining the same results. In addition, the approach offers the possibility of practicing online learning, a technique in which training can be paused and continued with another dataset. Future work consists of extending the approach with other NNs.

In [71], Raedler et al. present a SysML-based approach allowing to graphically model machine learning algorithms. The approach is based on hierarchical organized stereotypes that are used to detail either functions or artifacts. Due to the integration in SysML, the approach is well aligned with CRISP-DM. Based on the stereotypes and the mapping of stereotypes to implementation templates, the approach leverages on model transformation to streamline the imple-

mentation of machine learning algorithms [71]. Future work includes the systematic backflow of information and the extension of the approach to be more readily applicable.

An MDE approach for defining dataset requirements is introduced in [72]. It focuses on the structural definition of requirements using semiformal modeling techniques. Future work of the approach aims in rigorous validation of the models and adaption by model-to-model transformation improvement.

4.1 Model-driven engineering concerns

In this section, we report the contributions of the selected studies with respect to MDE techniques and practices [13, 21].

In this section, we analyze the adoption of MDE techniques and practices by the selected studies on several key aspects:

- **Language Workbench (LW) Adoption:** We explore whether LWs have been utilized for metamodel (*MM*) or grammar (*G*) specifications. Specifically, we investigate whether textual (*txt*) and/or graphical (*g*) concrete syntax (*CS*) is available within these workbenches.
- **Validation (V):** We assess whether the studies consider validation routines via model constraint definitions or model transformations to other engineering artifacts.
- **Model Transformations (MTrafo):** We investigate whether the proposed MDE approaches include model transformations

- **Applicability at Development (*d*) and/or Runtime (*r*):** We examine whether the proposed approach is applicable during development (design and implementation) and/or runtime (execution).

The analysis results are presented in Table 5.

Language workbenches, metamodels, and grammars

With the term language workbench, we refer to a tool or set of tools designed for software development within the language-oriented programming paradigm. It typically includes tools to support the definition, reuse, and composition of DSLs through a dedicated integrated development environment [47].

All the selected studies present metamodels, grammars, or UML profiles to define the concepts of the proposed DSLs.

In particular, the proposed DSLs have been developed using the following LWs.

EMF based. In this set, we consider approaches directly relying on the EMF LW. Therefore, in addition to approaches proposing DSLs based on Ecore-based metamodels, it includes EMF-based approaches like Xtext-based grammars and UML profiles. In 10 studies, the metamodel is based on EMF [2, 9, 26, 45, 52, 61–63, 71, 72]. Several EMF metamodels focus on the description of datasets [2, 26, 72]. Other studies additionally describe algorithms [2, 52, 61] or even further steps of the implementation [45, 63]. [9], three Xtext-based DSLs for data description, input and output data definitions for machine learning processes, and to configure optimization algorithms. In [52], the conceptual metamodel presented as an entity-relationship diagram in [14] is realized as a UML profile, i.e., a lightweight extension of the EMF-based UML metamodel in Papyrus. In [71], the metamodel is described using SysML stereotypes, an extension of Papyrus UML and, therefore, leverages lightweight extension of UML metamodel. Both [45, 62] extend an existing Xtext-based DSL, ThingML [40], with language constructs for AI/ML concerns.

KMF/Greycat-based. Two studies [41, 42] of the same research group are based on the Kevoree modeling framework (KMF) and its successor GreyCat, which results from a research project to create an alternative to the EMF based on Ecore. In [41], the capabilities of the Greycat metalanguage are presented. In particular, it allows the definition of microlearning units by explicitly declaring *learned attributes* as part of the domain-specific metamodels. In [42], Hartmann et al. introduce an exploratory approach proposing two metamodels for ML and meta-learning without specifically selecting a target LW. This work lacks an implementation. However, the authors explicitly suggest that both the Grey-

cat and EMF LWs are viable candidates for implementing their approach.

WebGME-based. Two studies [11, 43] define metamodels using the WebGME language workbench. While UML and profiles cannot provide the language engineering support typically offered by language workbenches, WebGME allows specifying DSLs, creating a class diagram-based metamodel from which the DSL infrastructure is automatically generated. In [11], the so-called STRATUM approach for BigData-as-a-Service provides a DSML consisting of several metamodels built on top of WebGME (metamodel for ML algorithms, metamodel for data ingestion frameworks, metamodel for data analytics applications, metamodels for heterogeneous resources). In [43], the metamodel is based on existing metamodel libraries: SEAM, DeepForge, and ROS-MOD.

Langium-based. In [37], the DescribeML DSL is the only work leveraging the recent Langium open-source language workbench enabling domain-specific languages in VS Code, Eclipse Theia, and web applications, leveraging the Language Server Protocol (LSP).¹¹ In [37], three metamodels are described i) metadata model, ii) composition model, and iii) provenance and social concerns model. Such metamodels are then implemented as grammars.¹²

MontiCore-based. In [55], all DSLs, i.e., MontiAnna, MontiAnnaTrain, and EmbeddedMontiArc, are all defined using the MontiCore language workbench [75]. One of the main benefit is the reuse of existing C++ code generators for neural network frameworks (MxNet, Caffe2, and Tensorflow).

MPS-based. In [60], five different DSLs are created with JetBrains MPS, an open-source projectional language workbench that allows direct changes to the abstract syntax tree through an editor, without the need for a grammar or parser. [60] leverages MPS' language extension and composition capabilities to deal with domain-independent (e.g., using the AdaptiveSystems DSL to structure the system according to MAPE-K loop by IBM) and domain-specific concerns (e.g., AdaptiveVLE to model concerns of virtual learning environments).

EBNF-based. In [66], the grammar of RADENN, a domain-specific language for neural networks, is defined using the extended Backus–Naur form (EBNF). The suite of tools for RADENN, developed in Python, facilitates the language's application. However, the methodology behind the toolset's creation—whether handcrafted or auto-generated via a Python-centric language workbench—remains unspecified (e.g., [27]).

¹¹ <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>

¹² Based on Chevrotain, <https://chevrotain.io/docs/>.

Concrete syntax

This section assesses the proposed approaches' notations or *concrete syntax*.

Ten studies [9, 26, 37, 41, 42, 45, 55, 60, 62, 66] provide a textual (or tabular) notation; six studies [11, 43, 52, 55, 63, 71, 72] adopt a graphical notation.

No concrete syntax available. Two studies [2, 61] do not provide a DSL-specific concrete notation. In particular, Al-Azzoni [2] left the definition of a complete DSL as future work while [61] is conceived to reuse the notations offered by tools defining data science pipelines. However, by leveraging EMF, a tree-based notation is possible by automatically generated editors, and, potentially, compatible technologies can provide textual or graphical concrete syntax options (e.g., via Xtext and Sirius, respectively). In Table 5, we reported the serialization format XMI as *default* notation since it can be used by users to inspect the model artifacts.

Textual notation. In [9, 26, 45, 62], all DSLs are defined in Xtext. [45, 62] extends the ThingML grammar, thus the generated artifacts are declared to be still compatible with the original Xtext-based textual editors. In [37], the textual concrete syntax is defined by a recent language workbench, Langium. In [41] and [42], an Emfatic-inspired textual modeling language is observed. In [60], five different interwoven DSLs are proposed, mixing textual and tabular projections, created with JetBrains MPS. In [66], Python-inspired textual notation is applied to model neural network concerns.

Graphical notation. In [11] and [43], the graphical concrete syntax is defined through capabilities offered by the WebGME language framework. [52] implements the meta-model as a UML profile in Papyrus. The UML class diagram is chosen as graphical notation since all the stereotypes inherit from the Class metaclass. No DSL-specific customization of the UML graphical notation is offered. [63] provides a web-based graphical editor realized using Sirius Web.¹³ In [72], the DSL provides a graphical concrete syntax and editor realized in Sirius.¹⁴ However, the paper does not discuss or show its graphical elements. In [71], graphical modeling using SysML and Papyrus is presented.

Multiple notations. In [55], Kushmenko et al. are the only ones proposing a mix of textual and graphical concrete notations to represent AI concerns. However, it is worth noting that the SVG-based hierarchical representation of components and connectors is made for visualization purposes and is not editable.¹⁵

Validation

Validation plays a pivotal role in MDE approaches. Modeling constraints assist modelers in creating valid models, which are essential for successfully executing automated engineering processes. While language constraints are typically enforced by metamodels and grammars, modelers can also define additional arbitrary constraints to perform more complex validation tasks on model artifacts. More sophisticated validation techniques, such as formal checks, may involve different artifacts, which are often woven together by model transformations.

In our research, we identified four studies that mention these constraints [2, 9, 61, 72].

Leveraging LW can yield a clear advantage in modeling and validation. LW can provide standards for constraint definitions languages like OCL and leverage ad hoc extensions of engineering platforms.

For example, the approach presented by Al-Azzoni [2] leveraged the Epsilon Validation Language (EVL) by the Epsilon to ensure the validity of learning problem and neural network models.

In [9], Berger et al. provide two separate Xtext-based data definition and constraint definition DSLs for textual specification of single and combined constraints on data, respectively.

In [61], constraints are required for satisfying reproducibility and replicability requirements of data science projects' pipelines. OCL constraints are mentioned even though not shown. Their approach leveraged EMF that provides OCL support for EMF-based metamodels (e.g., OCLinEcore¹⁶). Finally, in [72], the unified modeling language (UML) is employed for semiformal modeling. To validate formal constraints, the resulting models must be translated into Alloy for formal analysis. The Alloy language facilitates the validation of structural requirements for datasets in deep learning-based systems.

Model transformation

Fifteen selected studies explicitly mention or report model transformations as part of the proposed approaches. These model transformations are classified based on their intents, as described in [57], and the technology they use, as described in [48]. Table 6 summarizes the intents of the model transformation for each paper, as well as the main model-driven technologies used. It is important to note that none of the papers explicitly list or classify their model transformations. The identification of existing transformations and their intents is an attempt by the authors of this paper to provide a basis for comparison.

¹³ <https://www.eclipse.org/sirius/sirius-web.html>

¹⁴ <https://www.eclipse.org/sirius>

¹⁵ <https://github.com/EmbeddedMontiArc/Documentation>

¹⁶ <https://wiki.eclipse.org/OCL/OCLinEcore>

Eleven studies leverage model-to-code transformations [2, 11, 41–43, 45, 52, 60, 62, 71] to perform refinements on involved artifacts to generate executable code. Three studies [41, 43, 72] aim at executable models by defining translational semantics for their DSLs. Seven approaches [26, 41–43, 45, 66] provide more than one transformation with different intents. Two approaches [26, 37] translate artifacts across different modeling languages.

The rightmost column in Table 6 mentions the main model-driven technology leveraged by the studies to implement model transformations. The most commonly used platform among the studies is Eclipse, with Epsilon¹⁷ and Xtend¹⁸ being the most popular tools.

For example, in [2], the Epsilon Generation Language (EGL) is used in conjunction with templates to define model transformations that generate Java code. Similarly, [52] uses EGL to generate C# code for making predictions on test data. In [11], WebGME's code generation capabilities are extended with templates for each sub-task. In [26], two intents of model transformations are reflected: language translation and abstraction using a restrictive query. Based on Xtend, the model transformation transforms dataset descriptions into tabular datasets using low-level data transformation operations, which can then be used in data mining algorithms. In [41], the GreyCat framework, built on the KMF, provides code generation toolsets for building object-oriented applications. In [42], the concept of using code generators to generate ML code is mentioned. In [43], the ALC toolchain enables code generation for data collection or training exercises of learning-enabled components, as well as translational semantics for configuring an embedded Jupyter Notebook that executes the learning model. The approach also allows for the construction of safety cases. In [45], model-to-code transformations are implemented in Java within compilers that bind platform-independent AIoTML models to specific IoT platforms. In [55], the MontiAnna2X code generator generates MxNet, Caffe2, or Tensorflow code. In [60], JetBrains MPS language is used to generate Java code. In [62], Java and Xtend are used to generate Python code. In [72], model-to-code transformation is used to complete formal specifications using the Alloy analyzer. In [66], an interpreted language is presented. The approach first transforms the code into tokens and trees, further evaluating the syntax and interpreting the defined model tree. In [71], means of model transformation is used to generate executable code based on templates that are mapped with properties of the SysML model enriched with unambiguous stereotypes.

Development and runtime model

With term runtime model, [8] Bencomo et al. refer to a concept that involves the use of models during the runtime of a system to provide intelligent support and enable self-adaptation.

All the selected studies in this SLR propose model-driven approaches designed for use during various development stages.

Among the selected studies, Hartmann et al. [41] are the only that explicitly leverage runtime models. In their paper, they propose a design model that is instantiated at runtime, treated as an evolving object graph, and retains historical data. They introduce the concept of microlearning units, which are specified at development time within domain classes that lack a predefined behavioral model. These microlearning units are designed to learn and adapt at runtime based on the data associated with the object graph, effectively enabling the runtime model to acquire *known unknowns*, i.e., behaviors that were not predictable at design time but can be learned through live data.

4.2 Artificial intelligence concerns

Same as for the MDE concerns, the findings regarding AI development characteristics are presented in the following.

Business understanding

Industry often faces the problem of missing business understanding and shortcomings in elaborating business values [15, 16, 70, 83]. Therefore, modeling business understanding is essential for mature and comprehensive approaches, e.g., by defining requirements. The assessment revealed that five of the 18 approaches foster business understanding by integrating system-relevant modeling or processes.

In [43], the business understanding is fostered due to requirements and components modeling using SysML. Particularly, a goal structuring notation (GSN) approach is used to define and structure requirements.

In [63], business-relevant information is modeled through integrating *Roles*, leading to increased business understanding. Additionally, the metamodel reflects means to model requirements. However, details are currently missing on how the modeling is carried out.

In [45], the business understanding is defined by so-called things, which are constructs that are standard in the underlying concept of the modeling language ThingML. Particularly, devices, controllers, and simulators are defined as things.

In [72], a method to describe ML datasets from a requirements engineering perspective is presented. Notably, func-

¹⁷ <https://www.eclipse.org/epsilon>

¹⁸ <https://www.eclipse.org/xtend>

tional and nonfunctional requirements are integrated to describe dataset structural requirements.

In [71], business understanding is explicitly given by means of systems modeling, enabling to describe the origin of the data collected and utilized by the machine learning approach.

Data understanding

The data understanding fosters the downstream processes of CRISP-DM. Additionally, it allows assessing dataset quality and streamlining to form hypotheses for hidden information [87]. In the selected literature, 11 approaches support modeling some aspects of the data understanding.

[26] contextualizes dataset properties and improves data understanding by implicitly applying rules on how to select data. In [37], a detailed description of a dataset and data composition is given that fosters the overall data understanding. In [52], data understanding is enhanced due to the input data's graphical representation and the variables' composition. In [61], data understanding is promoted by describing data attributes such as the data type. Furthermore, the type of ML algorithm is described, allowing the reproduction of an ML project. In [63], data understanding is promoted by defining data attributes aligned with attribute types using UML class diagram.

In [41, 42], the enrichment of properties on a metamodel level is enabled, contributing to further description of the properties and, therefore, increasing data understanding. Moreover, the interconnection of the data properties is highlighted by the underlying principle. Still, the description of the attributes is not very detailed, leading to no support in understanding a single property and its origin. In [72], the advanced requirements modeling allows a better understanding of datasets with specific properties and structured data elements.

In [66], the data understanding can be modeled due to the build-in functions and the automatic type inference, which implicitly cast variables based on the use.

In [9], data understanding is the primary goal of the approach with separation of concerns of the various information suppliers.

In [71], data understanding is given as each property of the provided data is formalized and data types are specified using stereotypes.

Data ingestion

Eleven of the given 18 approaches describe the loading and ingestion of data, i.e., loading or referencing the input datasets.

In [60], the implementation of data ingestion using a DSL is described. Ten other approaches support the specification of a file path, URI, URL, etc., to reference data [2, 26, 43, 61, 63, 66, 71]. In [26], the loading of the dataset is described

by specifying the name and path of the file or SQL server in combination with SQL selection scripts. Therefore, this approach supports both file and database-related data. In [63], data loading from various sources, such as SQL servers, is supported.

In contrast, to fix data sources, the loading from edge devices or sensors is supported by three approaches [11, 55, 62]. In [11], data loading from various edge devices is presented using technologies such as RabbitMQ or Kafka. In [55], data loading is provided with tagging schemas for EMADL ports. In [62], two approaches are given, first a black-box approach, where the ML model is imported from a pickle, and second, the paths or URLs of the dataset(s) are passed to the training, validation, and testing of the algorithm.

Feature preparation

The preparation of features for certain ML algorithms is supported by eight of the 18 approaches.

In [11], the feature preparation is defined in the metamodel. Unfortunately, details on the specific methods, parameters, or the order of execution are missing. In [2], normalization of dataset features is supported. However, other preprocessing methods are not supported in the metamodel. In [61], data operations contain one or more input or output ports. Each data operation is an atomic operation on the input data to produce certain output data. In [62], each state allows executing functions. The keyword *DA_Preprocess* is used to apply data preparation methods on a specific dataset. In [45], feature preparation is given by the possibility to add additional layers and strategies of the image processing layers of the neural network.

In [63], features can be prepared with specific *feature extraction techniques*, and data can be transformed with data engineering techniques, e.g., *Regression substitution*. In [66], feature preparation is given due to the extendability of the build-in functions and due to the fact that datasets can be modified with the build-in functions. In [71], feature preparation is given with the application of stereotypes specifically applied to transform and arrange data attributes.

Model training

The specification of an algorithm and the related training of the model is depicted in 14 of the 18 approaches. The types of algorithms can be separated in Inference [52], Machine Learning [42, 60, 61, 63, 71] and Deep Learning [2, 11, 43, 45, 55, 62, 66] using neural networks.

Inference. [52] extended the approach of [14] with the required implementation using SysML and Papyrus modeling framework. Within the original approach [14], model training is given by an assignment for each variable, whether

it is an observed variable, a random variable, or a standard variable. Details on hyper-parameter tuning are not given.

Machine Learning. In [41, 42], various algorithm models can be used with specific input (learning) and output attributes. In [60], the algorithm (referred to as *approach*) is specified aligned with various hyper-parameters, e.g., random forest cross-validation folds. In [61], the algorithm type (e.g., random forest) with a specific task type (e.g., classification) can be described. Hyper-parameters are not presented in the metamodel. In [63], hyper-parameters and performance criteria can be specified for each AI model. In [71], algorithm type, parameters, and train/test splitting can be applied using stereotypes.

Deep Learning. In [2], the training is defined using an *MLPDescription* block with certain learning rules like back-propagation. Further details on other hyper-parameters or the output's facilitation are not given. In [11], an algorithm for the training is defined in the metamodel. Moreover, hyper-parameters are defined and applied to a specific algorithm in the editor. In [43], an experimental model defines the model training. The details of the implementation can be found in the Jupyter Notebooks. In [55], the training of NN is given with possibilities to specify the network layers and connections. In [62], state diagrams are used to define various steps of the algorithm. With the state keyword *DA_Train*, various training-related settings are made, and with *DA_Predict*, the trained model can be applied to data. In [45], the modeling of various layers is given by the extension of the metamodel and the possibility to define and deploy executable code on a simulation model.

In [66], the training is defined by creating input, hidden and output layer and executed with various parameter such as Epoch definition.

Metrics/evaluation

To assess the validity of an algorithm, six of the 18 approaches integrate the modeling of metrics.

In [43], the metrics are applied directly in the Jupyter Notebooks, which is not actually a modeling approach. Nevertheless, the Jupyter Notebook is integrated into the model. So it can be considered as part of the model.

In [11], metrics are integrated into the metamodel and can be applied to the training output. In [55], the evaluation metrics are selected using the name of the metrics, e.g., mean squared error (MSE).

In [62], basic metrics such as mean absolute error (MAE) or MSE can be applied to the algorithms, such as regression algorithms.

In [66], there are two approaches to apply metrics. For classification cases, average accuracy, macro-average sensitivity and macro averaging specificity are evaluated. For regres-

sion cases, the mean-square-error, mean-absolute-error, and coefficient of determination are evaluated.

In [71], various metrics such as mean-square-error or mean-absolute-error can be applied based on the stereotypes defined.

4.3 Frameworks (methods & tools)

In Table 7, we present a non-exhaustive list of building blocks (rows) corresponding to the approaches discussed in the selected studies (columns). Our categorization focuses on elements related to MDE and AI/ML. Specifically, we distinguish between language workbenches (LW), related languages (in addition to those proposed by the approaches themselves), and various tools.

Notably, several elements are reused across different approaches. In the MDE domain, language workbenches, such as EMF, Xtext, WebGME, and Greycat, play a significant role. Meanwhile, in the AI/ML domain, the Python programming language stands out as the most prevalent element.

4.4 Availability of artifacts and application domains

Artifacts serve as a means to facilitate the replication of research results. Table 8 indicates whether artifacts are referenced in the publication as online resources or are absent altogether. Additionally, the table depicts the type of application mentioned in the publication or inherently provided through the evaluation sample. If no specific domain is explicitly mentioned or derivable, we annotate it as *Unknown*. As a result, eight approaches work with datasets that can originate from any domain. The processing of internet of things (IoT) data is presented in five approaches, while one approach is more specific to image data.

5 Discussion

MDE has long been a cornerstone of software engineering research. Meanwhile, AI has witnessed remarkable advancements, with applications spanning various domains. Recently, the convergence of MDE and AI has emerged as a promising research direction. Figure 2 illustrates the publication trend of selected studies over time, emphasizing the certain interest in MDE4AI. We observe that the MODELS conference¹⁹ (with three publications) and the MODEL-SWARD conference²⁰ (with two publications) stand out as the most representative venues. However, it is essential to

¹⁹ <https://conf.researchr.org/series/models>

²⁰ <https://modelsward.scitevents.org/>

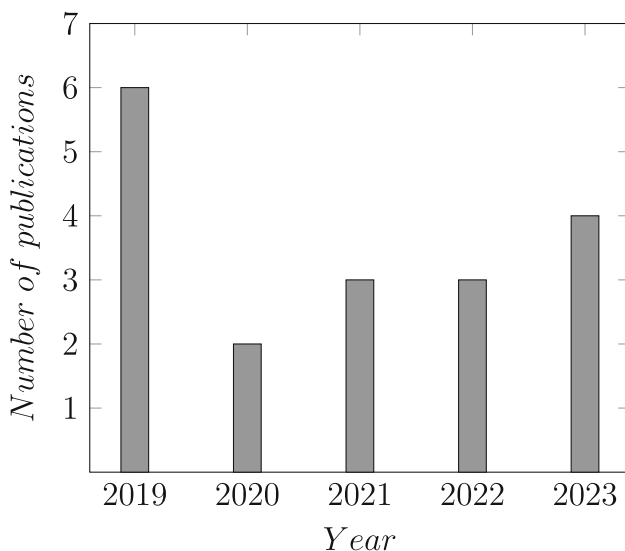


Fig. 2 Number of publication over year

acknowledge that the overall volume of collected publications remains limited.

This limitation may result from our selective exclusion criteria; our focus on MDE4AI led us to disregard AI4MDE, the complementary perspective.

From a research community perspective, the MDE research community remains keen on exploring this intersection further and establishing a focused community. Notably, the annual MDE Intelligence workshop at MODELS stimulates cross-pollination between MDE and AI.

The rest of the discussion is organized according to the research questions in Sect. 3.1.

5.1 RQ1—What language aspects of MDE are addressed in the approaches, e.g., abstract syntax, concrete syntax, metamodel, etc.?

- *Abstract Syntax:* Most MDE approaches explicitly address abstract syntax. It is a fundamental aspect because it forms the foundation for creating models. The abstract syntax is defined using metamodels and/or grammars, typically through a dedicated tool support of DSL development, a.k.a. language workbenches [47].
- *Concrete Syntax:* The results highlight that MDE approaches from select studies prefer textual modeling rather than graphical modeling. This preference may be influenced by tooling support, ease of manipulation, and expressiveness. It is essential to highlight a recent trend in MDE: the emergence of blended modeling capabilities. Blended modeling empowers engineers to freely select and switch between various notations for the same DSL. The choice of notation(s) can be influenced by diverse factors, including the complexity of graphical tools and the expertise of modelers.

- *Secondary aspects: Validation and Runtime Support:* Model validation ensures that models adhere to defined constraints and rules. Although this aspect is not a prominent focus in the selected studies, it is worth noting that language workbenches (LWs) provide mature support for model validation. Leveraging this support may lead to the development of more mature domain-specific languages (DSLs) and approaches. Runtime support is offered only by Hartmann et al. [41] at a metalanguage level through the GreyCat LW. However, this LW is not as common as other existing LWs. Therefore, we can argue that there is still room for improvement and support in this direction.

5.2 RQ2—Which phases of AI development aligned with the CRISP-DM methodology are covered by the approaches?

The CRISP-DM development cycle's supported phases are less balanced than the MDE perspectives. Less than half of the approaches support the early phases, such as business understanding. Feature preparation is often not mentioned or integrated with only simple features; e.g., normalization of variables is given but not the subsequent processing of preprocessing tasks. The main focus of the approaches lies in the formalization of model training. However, most of the approaches only support a small range of algorithms. Therefore, the applicability might be very case-specific and less flexible.

In summary, it can be seen that multiple approaches depict a specific aspect of the CRISP-DM development cycle, but only a few support more than half of the phases.

5.3 RQ3—Which industrial domains are supported by MDE4AI approaches?

Most approaches support processing datasets in specific file formats or using data from SQL servers. Since these datasets can originate from any domain, no focus on a domain can be determined in these approaches.

However, some approaches are rather based on IoT/CPS or sensor data, supporting the integration of production systems or data from the use of e.g., CPS products. Nevertheless, no domain can be clearly defined here since collecting sensor data is possible in any domain.

5.4 RQ4—What are the used methods and MDE tools the proposed approaches rely on?

The present works are based on a wide variety of tools and methods. Therefore, the advantages and disadvantages of the individual methods and tools are considered application-dependent, and no statement can be made about the quality of

the underlying methods. Furthermore, there is a trend toward Eclipse and its products (Papyrus, Sirius, Epsilon, etc.).

The adoption of language workbenches (LWs) appears to be crucial. LWs facilitate the creation of textual and graphical concrete syntaxes, essential for model representation and manipulation. Using LWs, such as EMF, Xtext, WebGME, and Greycat, underscores the importance of standardized tools in model-driven engineering (MDE).

EMF and Xtext can also be identified as state of the art for defining metamodels and grammars or as a basic modeling construct. Adopting other LWs seems more innovative, but it is still limited to a smaller community of researchers.

However, it is worth noting that Greycat seems to be a niche solution, with both uses originating from the same research group. We notice a shift of Greycat features toward being offered as a commercial tool, raising questions about its future adoption within the research community.

MontiCore, a well-known platform in the MDE community [75], is expected to play an increasing role in the intersection between MDE and the AI/ML domain.

Langium, a recent LW based on TypeScript, is particularly suitable for integrating web-based tools and applications, and, therefore, to foster collaborative activities and promote the use of web-based tools with lower installation costs. Unlike Xtext, Langium does not rely on the Eclipse Modeling Framework (EMF).

5.5 RQ5—To what extent is communication between different stakeholders supported by MDE

Unifying the language of communication can foster communication in an AI project, potentially leading to a better understanding and reduced unknown knowledge among team members. With less unknown knowledge, unrealistic expectations might be reduced, being one of the categories of why AI projects fail [86]. The intersection with other domains is mainly in the initial phases of an AI project, mainly business and data understanding. Still, the documentation of other phases of the CRISP-DM cycle supports communication among other AI experts. Only five approaches to interdisciplinary communication support the documentation and integration of business understanding, leading to further research needs. Data understanding and the downstream processes of the CRISP-DM are more often supported. However, still, further integration of MDE techniques is required due to the early development of some of the approaches.

5.6 RQ6—Which challenges and research directions are still open?

The researchers' observations guide the direction of future research and highlight open challenges. The first observation

emphasizes the need for better support in business understanding.

In the literature, experts report that the lack of business values for AI is a challenge. This deficiency may stem from AI experts' limited understanding of specific business contexts. Consequently, AI experts might not recommend realistic and relevant AI approaches.

Aligned with business understanding, project requirements should be formalized. This formalization enables the derivation of project metrics and facilitates assessing the impact of computational support [73].

Furthermore, considering that the second-largest group of supported applications in existing works is IoT. Therefore, systems engineering requires to be considered more in MDE4AI approaches.

Another future work that supports the maturity of MDE4AI is consolidating the advantages of the existing approaches and extending these approaches to fit various use cases. Combining various approaches to a comprehensive methodology regarding MDE4AI could streamline the research topic and foster the development of MDE4AI toolboxes.

It is worth noting that this study did not identify any valuable contributions, despite the recent trends in low-code and no-code development platforms targeting intelligent systems. One possible reason could be the complexity of the engineering process for AI software systems, which remains too high for platforms aiming at the highest process automation. Consequently, we anticipate more valuable contributions in AI4MDE specifically tailored for low-code/no-code platforms. Such contributions would cater to the needs of citizen developers who can leverage intelligent chatbots (e.g., [24]) to accomplish platform tasks.

Apart from combining the research workforces, future research needs to focus on engineering collaboration using methods designed for collaborative modeling. In this regard, the WebGME and Langium LWs are valuable MDE assets. Concerning more extensive or interdisciplinary projects, the live or collaborative work on a single model could increase the development performance and the benefit and acceptance of MDE4AI.

Next, the output of MDE4AI is often derived from Python code. Python is an easy-to-understand, well-known, daily-used language used by AI experts that might lead to changes in the Python code rather than the model. Consequently, full code generation is not applied, leading to no single source of information because partial truth of information is stored in the model and partial in the Python code [13]. In this context, it is necessary to elaborate a closed-loop process that feeds the results of the executed algorithm back into the model or adjusts the model in case of changes in the code, e.g., in Python. With this closed-loop approach, the model is always up-to-date, and further, collaboration with others potentially

improves because of the abstract representation of the actual changes.

Finally, only a few approaches mention user studies to assess the impact and benefits of MDE4AI. User studies are required to identify unused potentials and further streamline the development toward a user-centered MDE4AI methodology.

Although the conclusion of this research question suggests many improvements, it is questionable whether all the challenges of MDE4AI can be resolved under a single model, method, or framework. Therefore, future work needs to investigate whether a separation of concerns, e.g., based on purpose, AI technology such as machine learning, deep learning, etc., might be more valuable and push the development of MDE in the domain of AI further. Additionally, this can shape the details on the benefits that MDE can bring to AI.

6 Threats to validity

The study's validity describes the extent to which the results are trustworthy and how biases arising from the subjective views of the researcher are avoided during the analysis. Validity must be considered at all stages of a study, and several approaches have been proposed in the literature. Following [50], the following threats to validity are considered:

- **Construct Validity:** Construct validity describes the validity of the concept or theory behind the study design such that the results are generalizable [88]. In this SLR, construct validity refers to the potentially subjective analysis of the studies and the different ways in which data extraction is conducted. Following the guidelines [50], each study analysis is conducted independently by at least two researchers. If the researchers cannot agree on a conclusion, a third researcher evaluates and discusses the literature until there is no disagreement. In addition, each selected literature was evaluated using the quality criteria suggested by [56]. A protocol based on [50] was defined for performing the extraction protocol, which was discussed by the performing researchers after each step.
- **Internal Validity:** Internal validity describes the causal relationships of the researcher's investigation of whether a factor influences an aspect under study. The particular danger is that a third factor has an unknown effect or side effect. To avoid this danger, the same behavior as for construct validity applies, that more than one researcher assesses the causal relationships. In addition, the tactic suggested by [50] was followed.
- **External Validity:** External validity exists when a finding in the selected literature is of interest to others outside the case under study. In this regard, the SLR uses a quality assessment based on [56], so included papers are published in peer-reviewed. Therefore, third-party inves-

tigators pre-assessed the selected studies, and the validity of the initial publication is the responsibility of the external authors.

- **Conclusion Validity:** The validity of the conclusion relates to concerns about the reproducibility of the study. The concerns in this paper relate to the possible omission of studies. In this regard, the concerns are mitigated by the carefully applied search strategy using multiple digital libraries in conjunction with the snowballing system as per [50]. In addition, the researchers followed the detailed search protocol as defined in Sect. 3 and applied the quality ratings. However, some concerns might exist due to the interdisciplinary nature of the fields involved and the various definitions of modeling and AI, and the overall complexity of the field. The authors tried to mitigate this problem by discussing the keywords with experts in the field, checking various publication venues in different research areas and clearly defining the naming conventions in the background section.

7 Conclusion

AI is emerging in several disciplines today and has recently attracted the interest of the MDE community, with several workshops being held on the subject. The development of AI requires several development phases, which potentially can be supported using MDE approaches. Currently, the support of AI by MDE is still at an early stage of development. Therefore, it is necessary to understand the existing approaches to support AI to streamline future research and build on existing knowledge.

We conducted an SLR to investigate the existing body of knowledge in MDE approaches to formalize and define AI applications. To this end, we followed a rigorous, SLR protocol, selected 18 approaches, and evaluated them for several dimensions of interest, from MDE and AI.

The result showed that the language engineering perspective of MDE for AI is already mature, and some approaches seem applicable in industrial case studies. The MDE approaches focus on the training phase of the AI approaches, while time-consuming tasks such as data preprocessing are not considered often. Additionally, the focus is not on improving communication, collaboration, or understanding of the business processes to be supported, which is reported in the literature as a core problem in AI development projects. Finally, the review showed that the approaches are case-specific and lack general applicability.

Future work in this research area consists of, among other things, consolidating approaches to combine benefits, expanding approaches to be less case-specific, and adopting a closed-loop process that allows for model-based development that potentially leads to an authoritative source of truth.

Funding Open access funding provided by TU Wien (TUW).

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Akkiraju, R., Sinha, V., Xu, A., Mahmud, J., Gundecha, P., Liu, Z., Liu, X., Schumacher, J.: Characterizing Machine Learning Processes: A Maturity Framework. In: Fahland, D., Ghidini, C., Becker, J., Dumas, M. (eds.) *Business Process Management*, pp. 17–31. Springer, Cham (2020)
2. Al-Azzoni, I.: Model Driven Approach for Neural Networks. In: 2020 International Conference on Intelligent Data Science Technologies and Applications (IDSTA), pp. 87–94, (2020)
3. Atouani, A., Kirchhof, J.C., Kusmenko, E., Rumpe, B.: Artifact and reference models for generative machine learning frameworks and build systems. In: *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences, GPCE 2021*, pp. 55–68, New York, NY, USA, (2021). Association for Computing Machinery
4. Azevedo, A., Santos, M.F.: KDD, SEMMA and CRISP-DM: a parallel overview. In: *IADIS European Conference Data Mining*, pp. 182–185, (2008)
5. Basili, V.R., Caldiera, G., Rombach, H.D.: The Goal question metric approach. pp. 1–10, (1994)
6. Baumann, N., Kusmenko, E., Ritz, J., Rumpe, B., Weber, M.B.: Dynamic data management for continuous retraining. In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS '22*, pp. 359–366, New York, NY, USA, (2022). Association for Computing Machinery
7. Beihoff, B., Oster, C., Friedenthal, S., Paredis, C., Kemp, D., Stoewer, H., Nichols, D., Wade, J.: A World in motion—systems engineering vision 2025. In: *Technical report, INCOSE*, San Diego, California, (2014)
8. Bencomo, N., Götz, S., Song, H.: Models@ run. time: a guided tour of the state of the art and research challenges. *Softw. Syst. Model.* **18**, 3049–3082 (2019)
9. Berger, B.J., Plump, C., Drechsler, R.: EVOAL: a domain-specific language-based approach to optimisation. In: *2023 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–10, Chicago, IL, USA, (July 2023). IEEE
10. Bertolini, M., Mezzogori, D., Neroni, M., Zammori, F.: Machine Learning for industrial applications: a comprehensive literature review. *Expert Syst. Appl.* **175**, 114820 (2021)
11. Bhattacharjee, A., Barve, Y., Khare, S., Bao, S., Kang, Z., Gokhale, A., Damiano, T.: STRATUM: a BigData-as-a-service for lifecycle management of IoT analytics applications. In: *2019 IEEE International Conference on Big Data (Big Data)*, pp. 1607–1612, (2019)
12. Bourque, P., Dupuis, R.: *Software engineering body of knowledge (swebok)*. IEEE Computer Society, EUA (2004)
13. Brambilla, M., Cabot, J., Wimmer, M.: *Model-Driven Software Engineering in Practice. Synthesis Lectures on Software Engineering*, vol. 1, 2nd edn., pp. 1–207. Morgan & Claypool Publishers, Williston (2017)
14. Breuker, D.: Towards model-driven engineering for big data analytics—an exploratory analysis of domain-specific languages for machine learning. In: *47th Hawaii International Conference on System Sciences, HICSS 2014, Waikoloa, HI, USA, January 6–9, 2014*, pp. 758–767, (2014)
15. Brunnbauer, M., Piller, G., Rothlauf, F.: Idea-AI: developing a method for the systematic identification of AI use cases (2021)
16. Brunnbauer, M., Piller, G., Rothlauf, F.: Top-down or explorative? A case study on the identification of AI use cases. (2022)
17. Brunton, S.L., Kutz, J.N., Manohar, K., Aravkin, A.Y., Morgansen, K., Klemisch, J., Goebel, N., Buttrick, J., Poskin, J., Blom-Schieber, A.W., Hogan, T., McDonald, D.: Data-driven aerospace engineering: reframing the industry with machine learning. *AIAA J.* **59**(8), 2820–2847 (2021)
18. Bucaioni, A., Cicchetti, A., Ciccozzi, F.: Modelling in low-code development: a multi-vocal systematic review. *Softw. Syst. Model.* **21**(5), 1959–1981 (2022)
19. Burgueño, L., Cabot, J., Wimmer, M., Zschaler, S.: Guest editorial to the theme section on AI-enhanced model-driven engineering. *Softw. Syst. Model.* **21**(3), 963–965 (2022)
20. Burgueño, L., Burdusel, A., Gérard, S., Wimmer, M.: MDE Intelligence 2019: 1st Workshop on Artificial Intelligence and Model-Driven Engineering. In: *Proceedings of the 22nd International Conference on Model Driven Engineering Languages and Systems, MODELS '19*, pp. 168–169. IEEE Press, (2021)
21. Burgueño, L., Ciccozzi, F., Famelis, M., Kappel, G., Lambers, L., Mosser, S., Paige, R.F., Pierantonio, A., Rensink, A., Salay, R., Taentzer, G., Vallecillo, A., Wimmer, M.: Contents for a model-based software engineering body of knowledge. *Softw. Syst. Model.* **18**(6), 3193–3205 (2019)
22. Burgueño, L., Kessentini, M., Wimmer, M., Zschaler, S.: MDE Intelligence 2021: 3rd Workshop on Artificial Intelligence and Model-Driven Engineering. In: *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 148–149, (2021)
23. Czarnecki, K.: Overview of generative software development. In: *Unconventional Programming Paradigms: International Workshop UPP 2004, Le Mont Saint Michel, France, September 15–17, 2004, Revised Selected and Invited Papers*, pp. 326–341. Springer, (2005)
24. Daniel, G., Cabot, J., Deruelle, L., Derras, M.: Xatkit: a multimodal low-code chatbot development framework. *IEEE Access* **8**, 15332–15346 (2020)
25. Davey, C., Friedenthal, S., Matthews, S., Nichols, D., Nielsen, P., Oster, C., Riethle, T., Roedler, G., Schreinemakers, P., Sparks, E., Stoewer, H.: Systems engineering vision 2035—engineering solutions for a better world. In: *Technical report, INCOSE*, San Diego, California (2022)
26. de la Vega, A., García-Saiz, D., Zorrilla, M., Sánchez, P.: Lavoisier: a DSL for increasing the level of abstraction of data selection and formatting in data mining. *J. Comput. Lang.* **60**, 100987 (2020)
27. Dejanović, I., Vadera, R., Milosavljević, G., Vuković, Ž.: Textx: a python tool for domain-specific languages implementation. *Knowl.-Based Syst.* **115**, 1–4 (2017)
28. DeLine, R.A.: Glinda: Supporting Data Science with Live Programming, GUIs and a Domain-specific Language. In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–11, Yokohama Japan, (2021). ACM
29. Di Ruscio, D., Kolovos, D., de Lara, J., Pierantonio, A., Tisi, M., Wimmer, M.: Low-code development and model-driven engineering: two sides of the same coin? *Softw. Syst. Model.* **21**(2), 437–446 (2022)
30. Di Sipio, C., Di Rocco, J., Di Ruscio, D., Nguyen, P.T.: Morgan: a modeling recommender system based on graph kernel. *Softw. Syst. Model.* **22**(5), 1427–1449 (2023)

31. Dogan, A., Birant, D.: Machine learning and data mining in manufacturing. *Expert Syst. Appl.* **166**, 114060 (2021)
32. Emmert-Streib, F., Dehmer, M.: Defining data science by a data-driven quantification of the community. *Mach. Learn. Knowl. Extr.* **1**(1), 235–251 (2019)
33. Espadinha-Cruz, P., Godina, R., Rodrigues, E.M.G.: A review of data mining applications in semiconductor manufacturing. *Processes* **9**(2), 1–38 (2021)
34. Fahle, S., Prinz, C., Kuhlenkötter, B.: Systematic review on machine learning (ML) methods for manufacturing processes—identifying artificial intelligence (AI) methods for field application. *Procedia CIRP* **93**, 413–418 (2020)
35. Forootan, M.M., Larki, I., Zahedi, R., Ahmadi, A.: Machine learning and deep learning in energy systems: a review. *Sustainability (Switzerland)* **14**(8), 4832 (2022)
36. Fowler, M.: *Domain Specific Languages*, 1st edition Addison-Wesley Professional, Boston (2010)
37. Giner-Miguel, J., Gómez, A., Cabot, J.: Describeml: a tool for describing machine learning datasets. In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: companion Proceedings, MODELS '22*, pp. 22–26, New York, NY, USA, (2022). Association for Computing Machinery
38. Giray, G.: A software engineering perspective on engineering machine learning systems: state of the art and challenges. *J. Syst. Softw.* **180**, 111031 (2021)
39. Goodfellow, I.J., Bengio, Y., Courville, A.: *Deep learning*. MIT Press, Cambridge (2016)
40. Harrand, N., Fleurey, F., Morin, B., Husa, K.E.: Thingml: a language and code generation framework for heterogeneous targets. In: *Proceedings of the ACM/IEEE 19th international conference on model driven engineering languages and systems*, pp. 125–135, (2016)
41. Hartmann, T., Moawad, A., Fouquet, F., Le Traon, Y.: The next evolution of MDE: a seamless integration of machine learning into domain modeling. *Softw. Syst. Model.* **18**, 1285–1304 (2019)
42. Hartmann, T., Moawad, A., Schockaert, C., Fouquet, F., Traon, Y.L.: Meta-modelling meta-learning. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pp. 300–305, (2019)
43. Hartsell, C., Mahadevan, N., Ramakrishna, S., Dubey, A., Bapty, T., Johnson, T., Koutsoukos, X., Sztipanovits, J., Karsai, G.: Model-based design for CPS with learning-enabled components. In: *Proceedings of the Workshop on Design Automation for CPS and IoT, DESTION '19*, pp. 1–9, New York, NY, USA, (2019). Association for Computing Machinery
44. Henderson, K., Salado, A.: Value and benefits of model-based systems engineering (MBSE): evidence from the literature. *Syst. Eng.* **24**(1), 51–66 (2021)
45. Ming, H., Cao, E., Huang, H., Zhang, M., Chen, X., Chen, M.: AIoTml: a unified modeling language for AIoT-based cyber-physical systems. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **42**(11), 3545–3558 (2023)
46. Hultdt, T., Stenius, I.: State-of-practice survey of model-based systems engineering. *Syst. Eng.* **22**(2), 134–145 (2019)
47. Jung, A., Carbonell, J., Marchezan, L., Rodrigues, E., Bernardino, M., Basso, F.P., Medeiros, B.: Systematic mapping study on domain-specific language development tools. *Empir. Softw. Eng.* **25**(5), 4205–4249 (2020)
48. Kahani, N., Bagherzadeh, M., Cordy, J.R., Dingel, J., Varró, D.: Survey and classification of model transformation tools. *Softw. Syst. Model.* **18**(4), 2361–2397 (2019)
49. Kelly, T., Weaver, R.: The goal structuring notation—a safety argument notation. In: *Proceedings of the dependable systems and networks 2004 workshop on assurance cases*, vol. 6. Citeseer Princeton, NJ, (2004)
50. Kitchenham, B., Charters, S.: Guidelines for performing systematic literature reviews in software engineering. In: *Technical report*, (2007)
51. Kitchenham, B., Brereton, P.: A systematic review of systematic review process research in software engineering. *Inf. Softw. Technol.* **55**(12), 2049–2075 (2013)
52. Koseler, K., McGraw, K., Stephan, M.: Realization of a machine learning domain specific modeling language: a baseball analytics case study. In: Hammoudi, S., Pires, L.F., Selic, B. (eds.) *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development, MODELSWARD 2019*, Prague, Czech Republic, February 20–22, 2019, pp. 13–24. SciTePress, (2019)
53. Kourouklidis, P., Kolovos, D., Matragkas, N., Noppen, J.: Towards a low-code solution for monitoring machine learning model performance. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS '20*, New York, NY, USA, (2020). Association for Computing Machinery
54. Kumar, P.S., Emfinger, W., Kulkarni, W., Karsai, G., Watkins, D., Gasser, B., Ridgewell, C., Anilkumar, A.: Rosmod: a toolsuite for modeling, generating, deploying, and managing distributed real-time component-based software using ros. In: *2015 International Symposium on Rapid System Prototyping (RSP)*, pp. 39–45, (2015)
55. Kusmenko, E., Pavlitskaya, S., Rumpe, B., Stuber, S.: On the Engineering of AI-Powered Systems. In: *2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW)*, pp. 126–133, San Diego, CA, USA, (November 2019). IEEE
56. Long, H.A., French, D.P., Brooks, J.M.: Optimising the value of the critical appraisal skills programme (CASP) tool for quality appraisal in qualitative evidence synthesis. *Res. Methods Med. Health Sci.* **1**(1), 31–42 (2020)
57. Lúcio, L., Amrani, M., Dingel, J., Lambers, L., Salay, R., Selim, G.M.K.K., Syriani, E., Wimmer, M.: Model transformation intents and their properties. *Softw. Syst. Model.* **15**(3), 647–684 (2016)
58. Madni, A., Purohit, S.: Economic analysis of model-based systems engineering. *Systems* **7**(1), 12 (2019)
59. Martínez-Fernández, S., Bogner, J., Franch, X., Oriol, M., Siebert, J., Trendowicz, A., Vollmer, A.M., Wagner, S.: Software engineering for ai-based systems: a survey. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **31**(2), 1–59 (2022)
60. Meacham, S., Pech, V., Nauck, D.: AdaptiveSystems: an integrated framework for adaptive systems design and development using MPS JetBrains domain-specific modeling environment. *IEEE Access* **9**, 127973–127984 (2021)
61. Melchor, F., Rodríguez-Echeverría, R., Conejero, J.M., Prieto, Á.E., Gutiérrez, J.D.: A Model-Driven Approach for Systematic Reproducibility and Replicability of Data Science Projects. In: Franch, X., Poels, G., Gailly, F., Snoeck, M. (eds.) *Advanced Information Systems Engineering, Lecture Notes in Computer Science*, pp. 147–163. Springer, Cham (2022)
62. Moin, A., Challenger, M., Badii, A., Günnemann, S.: A model-driven approach to machine learning and software modeling for the IoT: generating full source code for smart Internet of Things (IoT) services and cyber-physical systems (CPS). *Softw. Syst. Model.* **21**(3), 987–1014 (2022)
63. Morales, S., Clarisó, R., Cabot, J.: Towards a DSL for AI engineering process modeling. *Product-Focus. Softw. Process Improv.* **13709**, 53–60 (2022)
64. Naveed, H., Arora, C., Khalajzadeh, H., Grundy, J., Haggag, O.: Model driven engineering for machine learning components: a systematic literature review. *Inf. Softw. Technol.* **169**, 107423 (2024)

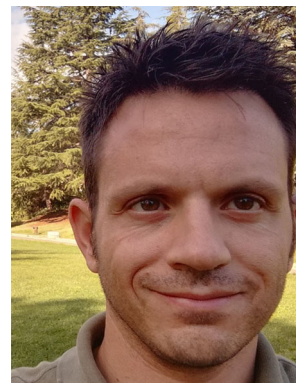
65. Petersen, K., Vakkalanka, S., Kuzniarz, L.: Guidelines for conducting systematic mapping studies in software engineering: an update. *Inf. Softw. Technol.* **64**, 1–18 (2015)
66. Pineda, I., Carrión-Ojeda, D., Fonseca-Delgado, R.: RADENN: a domain-specific language for the rapid development of neural networks. *IEEE Access* **11**, 86727–86738 (2023)
67. Piorkowski, D., Park, S., Wang, A.Y., Wang, D., Muller, M., Portnoy, F.: How AI developers overcome communication challenges in a multidisciplinary team: a case study. In: *Proceedings of the ACM on Human-Computer Interaction*, vol. 5(CSCW1), pp. 1–25, (2021)
68. Portugal, I., Alencar, P., Cowan, D.: A survey on domain-specific languages for machine learning in big data. (2016)
69. Provost, F., Fawcett, T.: Data science and its relationship to big data and data-driven decision making. *Big Data* **1**(1), 51–59 (2013)
70. Rädler, S., Rigger, E.: A survey on the challenges hindering the application of data science, digital twins and design automation in engineering practice. *Proc. Des. Soc.* **2**, 1699–1708 (2022)
71. Rädler, S., Rupp, M., Rigger, E., Rinderle-Ma, S.: Code generation for machine learning using model-driven engineering and SysML, (2023)
72. Ries, B., Guelfi, N., Jahic, B.: An MDE method for improving deep learning dataset requirements engineering using alloy and UML. In: Hammoudi, S., Pires, L.F., Seidewitz, E., Soley, R. (eds.) *Proceedings of the 9th International Conference on Model-Driven Engineering and Software Development, MODELSWARD 2021*, Online Streaming, February 8–10, 2021, pp. 41–52. SCITEPRESS, (2021)
73. Rigger, E., Vosgien, T., Shea, K., Stankovic, T.: A top-down method for the derivation of metrics for the assessment of design automation potential. *J. Eng. Des.* **31**, 1–31 (2019)
74. da Silva, A.R.: Model-driven engineering: a survey supported by the unified conceptual model. *Comput. Lang. Syst. Struct.* **43**, 139–155 (2015)
75. Rumpe, B., Hölldobler, K., Aachen, R.W.T.H. (eds.): *MontiCore 5 Language Workbench. Number Band 32 in Aachener Informatik-Berichte, Software-Engineering*. Shaker Verlag, Aachen, edition 2017 edition, (2017)
76. Sahay, A., Indamutsa, A., Di Ruscio, D., Pierantonio, A.: Supporting the understanding and comparison of low-code development platforms. In: *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 171–178, (2020)
77. Saltz, J.: CRISP-DM is still the most popular framework for executing data science projects, (November 2020)
78. Nelson Sanchez-Pinto, L., Luo, Y., Churpek, M.M.: Big data and data science in critical care. *Chest* **154**(5), 1239–1248 (2018)
79. Sarker, I.H.: Machine learning: algorithms, real-world applications and research directions. *SN Comput. Sci.* **2**(3), 160 (2021)
80. Schöne, R., Mey, J., Ren, B., Aßmann, U.: Bridging the gap between smart home platforms and machine learning using relational reference attribute grammars. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 533–542, (2019)
81. Schröder, C., Kruse, F., Gómez, J.M.: A systematic literature review on applying CRISP-DM process model. *Procedia Comput. Sci.* **181**, 526–534 (2021)
82. Shinde, P.P., Shah, S.: A review of machine learning and deep learning applications. In: *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBE)*, pp. 1–6, Pune, India, (August 2018). IEEE
83. Someh, I., Wixom, B., Zutavern, A.: Overcoming organizational obstacles to artificial intelligence value creation: propositions for research. pp. 5809–5818, (January 2020)
84. Trauer, J., Schweigert-Recksiek, S., Onuma, L., Spreitzer, K., Mörtl, M., Zimmermann, M.: Data-driven engineering—definitions and insights from an industrial case study for a new approach in technical product development. (2020)
85. Voelter, M., Benz, S., Dietrich, C., Engelmann, B., Helander, M., Kats, L.C.L., Visser, E., Wachsmuth, G.H.: *Dsl engineering-designing, implementing and using domain-specific languages*. (2013)
86. Westenberger, J., Schuler, K., Schlegel, D.: Failure of AI projects: understanding the critical factors. *Procedia Comput. Sci.* **196**, 69–76 (2022)
87. Wirth, R., Hipp, J.: CRISP-DM: towards a standard process model for data mining. In: *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, (2000)
88. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A.: *Experimentation in Software Engineering*. Springer, Berlin (2012)
89. Xu, J., Kovatsch, M., Mattern, D., Mazza, F., Harasic, M., Paschke, A., Lucia, S.: A review on AI for smart manufacturing: deep learning challenges and solutions. *Appl. Sci. (Switzerland)* **12**, 16 (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Simon Rädler is pursuing a doctorate at the Business Informatics Group at the Institute of Information Systems Engineering at the Vienna University of Technology (TUW) in Austria. Additionally, he is associated with the Chair of Information Systems and Business Process Management at the Faculty of Computer Science at the Technical University of Munich (TUM) in Germany. He holds a bachelor's and master's degrees in Computer Science. His research is at the intersection of

data engineering and model-driven engineering. In particular, he focuses on the integration of data engineering into model-driven engineering.



Luca Berardinelli is a senior researcher at the Business Informatics-Software Engineering department at JKU in Linz, Austria. His current research concerns the integration of Model-Driven Engineering, DevOps, and AI methodologies in the context of complex cyberphysical systems. He is participating in national and international funded projects on these topics. He was a postdoc at TU Wien, Austria, and the University of L'Aquila, Italy, where he received his MS and PhD degrees

in Computer Science.



Karolin Winter is an assistant professor in Information Systems at the Department of Industrial Engineering and Innovation Sciences at Eindhoven University of Technology (TU/e). Her main research interests are located in the area of business process compliance and process mining with a particular focus on applying, enhancing and developing process and data mining as well as natural language processing techniques.



His research activity concerns the integration of MDE, DevOps, and AI in the context of the European project AIDORt. His current research includes the application of AI in Model-Driven engineering. In particular, Abbas is interested in using generative AI (especially GANs) for data augmentation in the context of MDE.



Stefanie Rinderle-Ma is a full professor at the Technical University of Munich, Germany, and holds the Chair of Information Systems and Business Process Management. Her research interests focus on process-oriented information systems, flexible and distributed process technologies, compliance management, as well as production and process intelligence. The overarching goal of her research is to enable and accelerate digitalization and automation through processes and at the same time keep the human in the loop. Application areas comprise manufacturing, transportation and logistics, as well as medicine.